

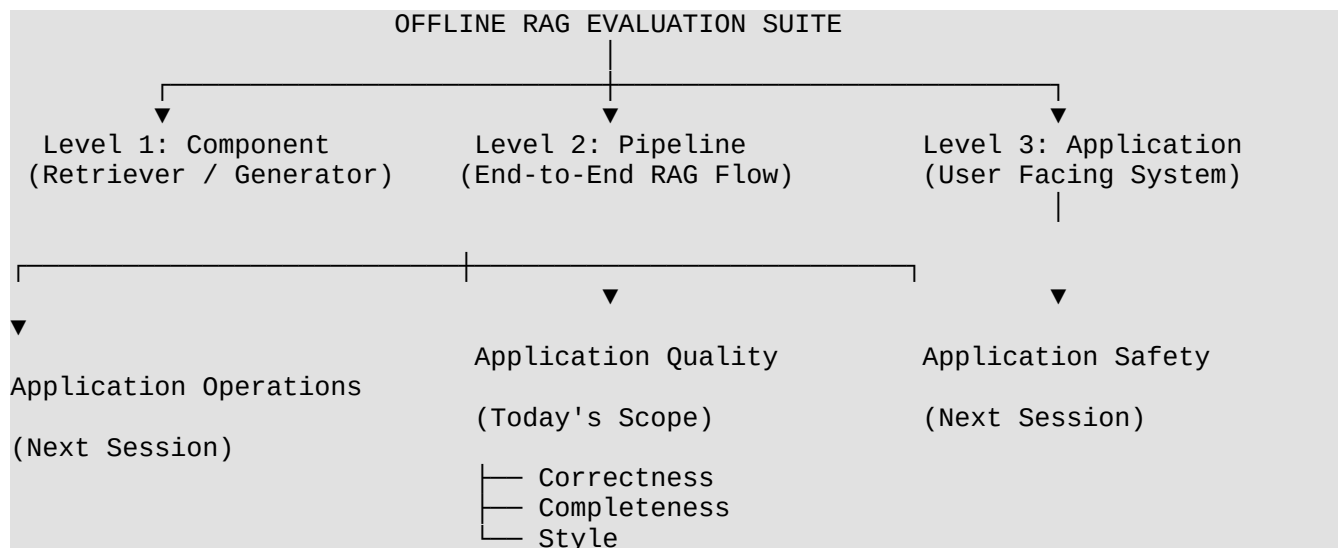
The Offline Evaluation Roadmap for RAG Applications

To understand our immediate objectives, we must first review our overall evaluation roadmap for building a production grade RAG pipeline. Our primary goal is to build an **Offline Evaluation Suite** structured across three distinct levels:

1. **Level 1: Component Level Evaluation** (assessing individual modules such as retriever and generator performance).
2. **Level 2: Pipeline Level Evaluation** (assessing end-to-end retrieval augmented generation behavior).
3. **Level 3: Application Level Evaluation** (assessing user facing application behavior).

Having completed component level and pipeline level evaluation suites in our previous sessions, we now advance to **Application Level Evaluation**. The Application Evaluation Suite is subdivided into three core operational pillars:

- **Application Quality:** Evaluating the core utility, factual truth, coverage and brand alignment of generated responses.
- **Application Safety:** Evaluating security risks, toxicity, hallucination bounds, and safety guardrails (scheduled for our next session alongside regression testing).
- **Application Operations:** Evaluating latency, cost, and system level throughput metrics (scheduled for our next session).



In today's session, we focus strictly on **Application Quality** across three primary metrics:

1. **Correctness:** Measuring whether the factual response generated by our RAG chatbot is correct or incorrect relative to universal truth.

- 2. **Completeness:** Measuring whether our chatbot provides a full response covering all sub questions within a multi part query, or merely returns a partial answer.
- 3. **Style:** Ensuring that the explanation style matches our specific **CampusX teaching voice**.

Analysis of Traditional Count Based Metrics

To appreciate why new evaluation paradigms are necessary for Application Quality, we must examine the five core evaluation metrics we previously established for RAG pipeline evaluation:

- 1. **Recall**
- 2. **Precision**
- 3. **Faithfulness**
- 4. **Answer Relevance**
- 5. **Context Relevance**

A structural pattern unites all five of these classical metrics: they are **Count Based Metrics**.

Operational Mechanics of Count Based Metrics

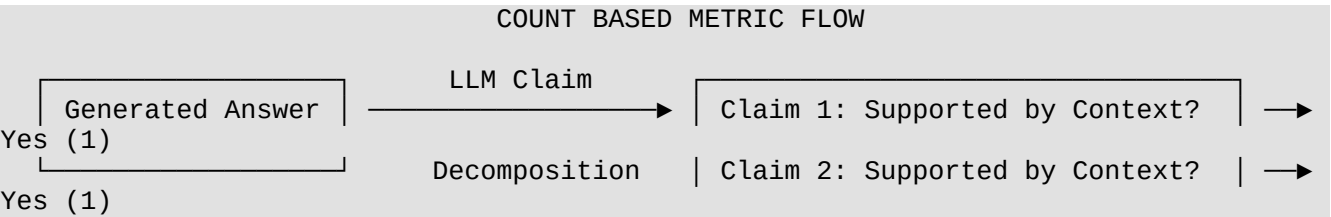
In a count based evaluation framework, metric computation follows a mechanical procedure based on claim extraction and mathematical ratio calculation.

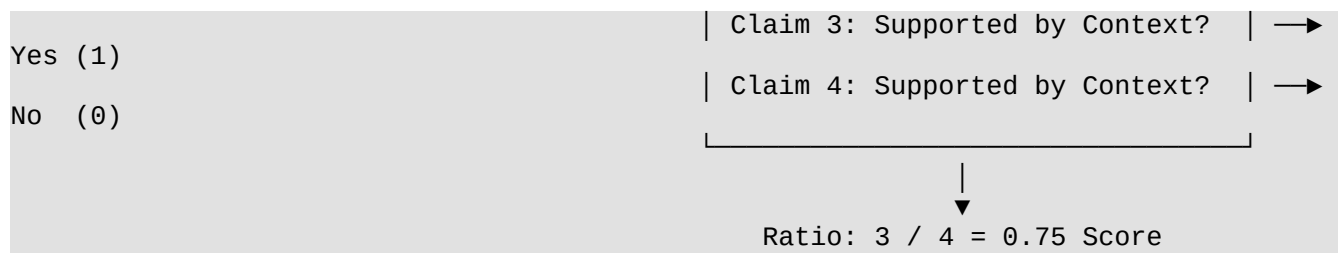
Consider **Faithfulness**, which measures whether a generated answer is derived exclusively from the retrieved context without inventing facts:

- 1. **Atomic Claim Breakdown:** We pass the generated answer to an LLM and instruct it to decompose the complete answer into discrete, standalone claims (C_1, C_2, C_3, C_4) .
- 2. **Binary Verification:** We take each extracted claim independently and ask an LLM whether that specific claim is explicitly supported by the retrieved context.
- 3. **Counting & Ratio Calculation:** We count the total number of claims supported by the context versus the total number of extracted claims.

$$\text{Faithfulness Score} = \frac{\text{Count of Context Supported Claims}}{\text{Total Extracted Claims}}$$

For example, if an answer is decomposed into 4 distinct claims, and 3 are present in the context while 1 is invented by the generator, the resulting Faithfulness score is exactly $(3/4 = 0.75)$.





Every classical metric Recall, Precision, Answer Relevance and Context Relevance relies on this identical formula: LLM based claim extraction, binary counting and ratio computation.

The Failure of Count Based Paradigms for Holistic Judgment Metrics

While count based extraction works for groundedness checks, it fails when applied to holistic, subjective, or structural metrics such as **Style**, **Correctness**, **Completeness**, **Helpfulness**, and **Safety**.

Scenario 1: Why Count Based Methods Fail for Style

Suppose we want to evaluate whether an answer follows the **CampusX teaching style**, which is defined pedagogically by explaining the **Why** (intuition), **What** (concept definition), and **How** (practical application).

If we attempt a count based approach, we would decompose the generated answer into isolated sentences and check if each sentence contains "Why, What, and How". This logic breaks down because pedagogical style does not exist inside individual atomic sentences, it emerges holistically across the complete response structure.

Scenario 2: Why Count Based Methods Fail for Correctness (The Analogy Problem)

Suppose we evaluate the factual **Correctness** of an answer by decomposing it into isolated claims and comparing each claim against a ground truth "golden answer".

Consider what happens when our RAG chatbot uses an **analogy** or a contextual example to explain a complex topic:

- An analogy uses a comparative example drawn from outside the immediate domain to build intuition.
- If an LLM judge evaluates that analogy statement as an isolated claim against a golden answer, the judge will observe that the analogy does not match the literal facts in the golden answer.
- Consequently, the judge flags the analogy statement as a false claim and penalizes the chatbot's correctness score, even though the explanation was correct and pedagogically sound.

Generated Answer: "Think of a database index like a library catalog..." |

↓
▼ (Isolated Claim Extraction)

Isolated Claim: "A database index is a physical library catalog."

↓
▼ (Compared against Golden Answer)

Golden Answer: "B Tree structure mapping keys to data page offsets."

↓
Result: LLM Judge marks claim as FALSE/UNRELATED → Unfair Score Deductions!

An answer cannot be judged on correctness or style by evaluating fragmented claims in isolation, it must be evaluated holistically as a unified output. This forces a paradigm shift from **Counting** (claim ratios) to **Judgment** (a human or LLM reading the entire response and assigning a score on a holistic scale, such as 0 to 10).

The Naive LLM-as-a-Judge Workflow

To evaluate judgment based metrics without relying on claim counting, we implement an **LLM-as-a-Judge** framework.

Operational Flow for Naive Correctness Evaluation

1. **Construct a Golden Dataset:** Create a benchmark dataset consisting of test questions paired with universally accepted, factually accurate **Expected Answers** (ground truth) authored by human domain experts (e.g., 15 test cases).
2. **Generate Actual Outputs:** Pass each test question through the RAG chatbot pipeline to obtain the **Actual (Generated) Answer**.
3. **Construct the Judge Prompt:** Pass the Question, Expected Answer, and Actual Answer into a direct evaluation prompt for an LLM judge:

System Prompt (Naive LLM-as-a-Judge):

"You are evaluating whether an AI's answer is correct.

You will be given a Question, an Expected Answer, and an Actual Answer.

Question: {question}

Expected Answer: {expected_answer}

Actual Answer: {actual_answer}

Compare the actual answer against the expected answer and decide how factually correct it is.

Give a score from 0 to 10 where 10 is fully correct and 0 is completely wrong."

1. **Aggregate Results:** Execute this judge call across all benchmark questions and compute the average score across the dataset.

Unlike classical metrics, where the LLM was used merely to extract claims for an explicit mathematical ratio, the naive LLM-as-a-Judge approach delegates the entire evaluation decision directly to the LLM's scoring output.

Structural Vulnerabilities and High Variance of Naive LLM-as-a-Judge

Although the naive LLM-as-a-Judge approach is straightforward to implement, it exhibits a critical failure in production: **High Score Variance and Instability**.

When running an identical evaluation setup repeatedly over unchanged data, score outputs fluctuate unpredictably across runs (e.g., swinging from 60% to 70% or 75%). In a reliable evaluation framework, identical inputs must yield stable, deterministic scores (e.g., minimal variance such as 84% to 85%).

NAIVE LLM-AS-A-JUDGE SCORE INSTABILITY		
Run 1 (Unchanged Code & Data)	→	Overall Score: 60%
Run 2 (Unchanged Code & Data)	→	Overall Score: 75%
Run 3 (Unchanged Code & Data)	→	Overall Score: 68%
Conclusion: HIGH VARIANCE makes evaluation UNRELIABLE for production engineering!		

This instability stems from two technical vulnerabilities:

Vulnerability 1: Unconstrained High Level Criteria

Providing a broad criteria statement ("decide how factually correct it is") leaves room for the model to interpret correctness differently across independent API calls. Without explicit guidelines ("constitution"), the judge model prioritizes different evaluation angles during each call, causing score shifts between evaluation runs.

Vulnerability 2: Categorical Integer Token Probability Flipping

Auto regressive language models generate text by computing a probability distribution (via softmax) over their output vocabulary for each token position.

When instructed to output a single score from 0 to 10, the candidate integer tokens (e.g., "7", "8", "6") receive the highest output probabilities.

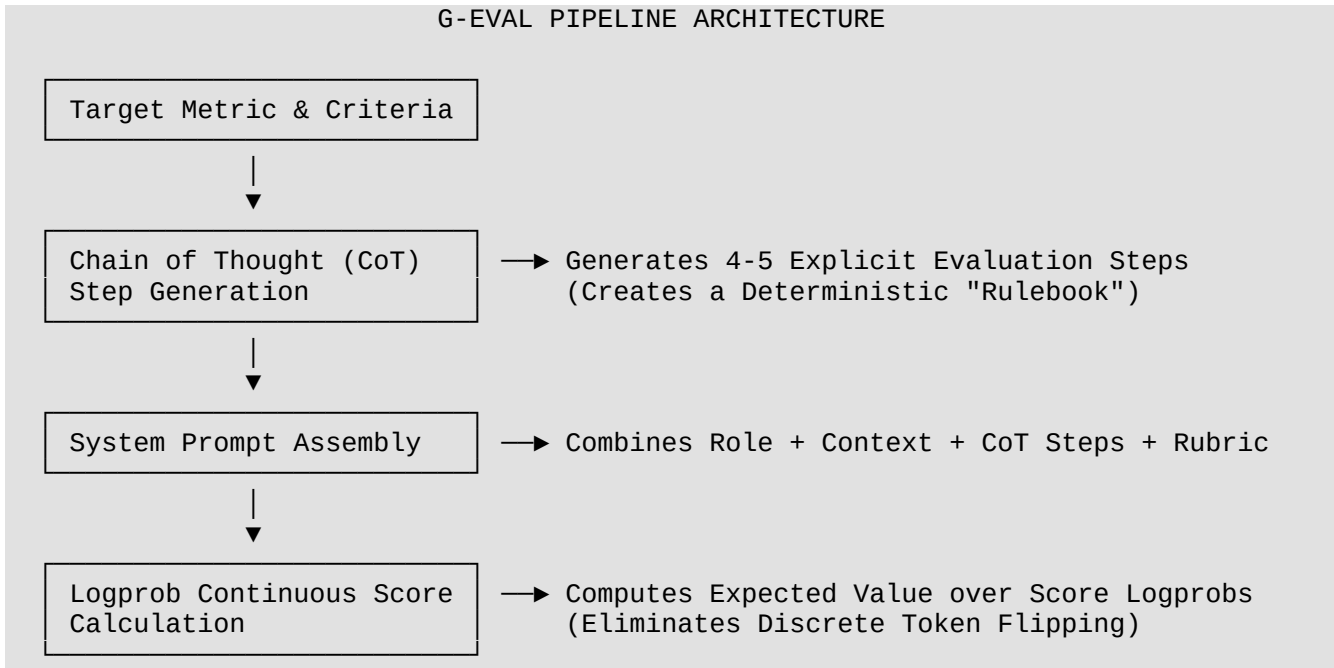
TRANSFORMER NEXT TOKEN PROBABILITY DISTRIBUTION		
Token Vocabulary	Run 1 Probability	Run 2 Probability (Slight Drift)
Token "8"	51% (Selected)	40%
Token "7"	40%	51% (Selected)

Token "6"	9%	9%
... [Other Tokens]	<1%	<1%

Because output selection relies on argmax token selection, a minor shift in probability (e.g., Token "8" moving from 51% to 40% while Token "7" moves from 40% to 51%) causes the output score to jump discretely from 8 to 7. Across a dataset of 15 questions, these discrete token flips cause large, unreliable overall score swings.

The G-Eval Framework Architecture

To resolve the instability of naive LLM-as-a-Judge setups, we implement **G-Eval**, a deterministic evaluation framework introduced in a 2023 research paper. G-Eval achieves high reliability by utilizing advanced judge models (such as GPT-4) combined with two structural innovations.



The Three Step Execution Architecture of G-Eval

- Metric & High Level Criteria Specification:** The developer specifies the target evaluation metric (e.g., Correctness) alongside a high level criteria definition.
- Chain of Thought (CoT) Step Breakdown:** G-Eval uses Chain of Thought prompting to decompose the high level criteria into 4 to 5 detailed, sequential evaluation steps, creating a deterministic "constitution" or rulebook.
- Prompt Assembly & Continuous Scoring:** The evaluation steps, inputs, expected outputs and an explicit **Scoring Rubric** are compiled into a comprehensive system prompt. Instead of taking the raw output token, G-Eval calculates a continuous, probability weighted continuous score from log probabilities.

G-Eval Core Innovation 1 Deterministic Evaluation Steps & Custom Steps

G-Eval's first core innovation is replacing broad criteria prompts with explicit, step by step evaluation instructions.

Consider a G-Eval evaluation prompt for Correctness:

Example G-Eval Evaluation Steps:

1. Compare only the factual claims in the Actual Output against the Expected Output.
2. A claim is wrong ONLY IF it contradicts the Expected Output or is factually false.
3. A factually accurate answer scores HIGH even if it is shorter or covers fewer points.
4. Do NOT deduct points for brevity or omitted points; only wrong statements count.
5. Additional correct information must NEVER lower the score.

By constraining the judge model to these explicit rules, we remove evaluation ambiguity across independent API calls.

Dynamic CoT Step Generation vs. Pre Defined Custom Steps

- **Dynamic CoT Step Generation:** When provided with only a high level criteria statement, G-Eval prompts GPT-4 internally to generate these evaluation steps on the fly.
- **Pre Defined Custom Steps:** Alternatively, we can bypass on the fly step generation by supplying hardcoded, custom evaluation steps directly to the framework.

DYNAMIC CoT VS. HARDCODED CUSTOM STEPS

Approach A: High Level Criteria → LLM CoT Generation → Dynamic Steps (Minor Variation)
Approach B: Hardcoded Custom Steps → Static Steps (Zero Variation)

Passing explicit custom steps eliminates step generation variance, ensuring the evaluation instructions remain identical across every run.

Optimal Engineering Workflow

1. **Exploration Phase:** During early pipeline design, pass high level criteria and allow the LLM to auto generate evaluation steps via CoT to discover error modes.
 2. **Production Phase:** Once failure patterns are understood, freeze the evaluation steps into explicit, hardcoded custom instructions and scoring rubrics to achieve maximum determinism.
-

G-Eval Core Innovation 2 Probability Weighted Continuous Score (Logprobs Expectation)

G-Eval's second innovation is its score calculation method, which addresses the token-flipping problem.

Instead of taking the single argmax integer token (e.g., token "8") from the model's output, G-Eval extracts the **log probabilities (logprobs)** of candidate numeric score tokens from the final layer softmax distribution.

LOGPROB EXTRACTION AND NORMALIZATION WORKFLOW		
Raw Model Softmax Output	Filtering & Extraction	Normalization
Token "8" : Prob 0.70 0.737	Token "8" : 0.70	P_norm(8) =
Token "7" : Prob 0.20 0.211	Token "7" : 0.20	P_norm(7) =
Token "9" : Prob 0.05 0.053	Token "9" : 0.05	P_norm(9) =
Token "The": Prob 0.03	[Filtered Out]	
Token ":" : Prob 0.02	[Filtered Out]	
	Sum = 0.95	

Mathematical Derivation of the Continuous Score

Step 1: Extract Top Score Token Probabilities

We query top output token probabilities for numeric score tokens $(S_1, S_2, \dots, S_k)$ and ignore non-numeric tokens (such as "The", ":").

Assume the extracted raw probabilities are: $P(\text{Token } 8) = 0.70, \quad P(\text{Token } 7) = 0.20, \quad P(\text{Token } 9) = 0.05$

Step 2: Normalize Probabilities Over the Candidate Score Space

Because non numeric tokens were removed, the raw probabilities sum to less than (1.0) (here, $(0.70 + 0.20 + 0.05 = 0.95)$). We normalize the remaining numeric token probabilities by dividing each by their sum:

$$P_{\text{norm}}(s) = \frac{P(s)}{\sum_{i \in \text{Scores}} P(i)}$$

$$P_{\text{norm}}(8) = \frac{0.70}{0.95} \approx 0.737, \quad P_{\text{norm}}(7) = \frac{0.20}{0.95} \approx 0.211, \quad P_{\text{norm}}(9) = \frac{0.05}{0.95} \approx 0.0526$$

Step 3: Compute the Expected Value (Weighted Average)

We compute the score as the expected value across normalized token probabilities:

$$\text{Continuous Score} = \sum_{s \in \text{Scores}} \left(s \times P_{\text{norm}}(s) \right)$$

$$\text{Continuous Score} = (8 \times 0.737) + (7 \times 0.211) + (9 \times 0.0526) = 5.896 + 1.477 + 0.4734 = 7.8464 \approx 7.85$$

If we had taken the standard argmax token selection, the output score would have been integer **8**. By computing the continuous probability weighted expectation, we obtain **7.85**.

Step 4: Scale to Benchmark Range & Pass/Fail Thresholding

The continuous score is normalized to a interval by dividing by 10 (\$7.85 / 10 = 0.785) and compared against a predefined pass threshold (e.g., 0.70).

When model confidence shifts slightly across runs, the weighted expectation moves smoothly (e.g., from 0.785 to 0.779), preventing discrete score jumps.

Section 9: Summary of G-Eval's Two Core Innovations

SUMMARY OF G-EVAL INNOVATIONS	
Innovation 1 Step-by-Step Evaluation Steps	Converts vague criteria into explicit CoT evaluation steps and scoring rubrics. (Eliminates criteria interpretation variance)
Innovation 2 Logprob Weighted Expectation	Replaces argmax integer token selection with probability-weighted continuous expectation. (Eliminates output token flipping variance)

These two innovations make G-Eval significantly more deterministic and stable than standard LLM-as-a-Judge setups.

Practical Implementation & Refinement of Application Quality Metrics

We implement our Application Quality Evaluation Suite in Python using the DeepEval library's GEval module.

```
# System Setup and Metric Import Configuration
from deepeval.metrics import GEval
from deepeval.test_case import LLMTestCaseParams, LLMTestCase
from deepeval.scenarios import Rubric
```

Our evaluation benchmark uses a golden dataset (correctness_goldens.json) containing 15 test questions paired with expert ground-truth answers.

Metric 1: Correctness Evaluation

Conceptual Distinction: Faithfulness vs. Correctness

- **Faithfulness:** Measures whether the generated output is grounded in retrieved context, regardless of whether that context is true in the real world.
- **Correctness:** Measures whether the generated output is factually accurate against real world truth and expert golden answers.

FAITHFULNESS VS. CORRECTNESS MATRIX			
Truth) Retrieved Context	Factually Correct (World Truth)		Factually Incorrect (World
	Grounded in	1. Faithful & Correct (Ideal Target State)	3. Faithful & Incorrect (Context contained errors)
Ungrounded in	2. Unfaithful & Correct (Answered from parametric memory	4. Unfaithful & Incorrect (Hallucinated false statement)	

Correctness Implementation (Iteration 1)

In our initial implementation, we define custom evaluation steps focusing heavily on key fact matching:

```
# Initial Correctness Implementation
correctness_metric = GEval(
    name="Correctness",
    evaluation_steps=[
        "Compare the actual output against key facts in expected output.",
        "Heavily penalize statements in actual output that contradict expected
output or are false.",
        "Reward statements matching expected output in meaning.",
        "Do not penalize for omitting information; only wrong statements count."
    ],
    evaluation_params=[
        LLMTestCaseParams.INPUT,
        LLMTestCaseParams.ACTUAL_OUTPUT,
        LLMTestCaseParams.EXPECTED_OUTPUT
    ],
    model="gpt-4o-mini",
    threshold=0.70,
    strict_mode=False # False enables probability-weighted continuous scoring
)
```

Iteration 1 Benchmark Result & Failure Analysis

Running Iteration 1 yields an overall Correctness score of **66%** (8 passed, 7 failed).

Analyzing the failure logs reveals the cause: human domain experts authored highly detailed golden answers. The RAG chatbot generated responses that were factually accurate but covered ~70% of the golden answer's depth. The judge model penalized these responses for incomplete coverage despite the evaluation step forbidding deductions for omitted points.

Correctness Refinement (Iteration 2)

To address this, we refine the evaluation steps to explicitly ignore brevity and add a strict **Scoring Rubric**:

```
# Refined Correctness Implementation with Explicit Rubric
correctness_metric_v2 = GEval(
    name="Correctness",
    evaluation_steps=[
        "Compare ONLY factual claims in actual output against expected output.",
        "A claim is wrong ONLY IF it contradicts expected output or is factually false.",
        "A factually accurate answer MUST score at least 9 even if shorter or covering fewer points.",
        "Do NOT deduct for brevity, omitted points, or missing elaboration; only wrong statements count.",
        "Additional correct information must NEVER lower score."
    ],
    rubric=[
        Rubric(score_range=(0, 4), explanation="Contains clear factual errors or direct contradictions."),
        Rubric(score_range=(5, 8), explanation="Mostly correct with 1-2 minor factual inaccuracies."),
        Rubric(score_range=(9, 10), explanation="All stated claims are completely factually accurate.")
    ],
    evaluation_params=[
        LLMTestCaseParams.INPUT,
        LLMTestCaseParams.ACTUAL_OUTPUT,
        LLMTestCaseParams.EXPECTED_OUTPUT
    ],
    model="gpt-4o-mini",
    threshold=0.70,
    strict_mode=False
)
```

Iteration 2 Results & Determinism Verification

Re-running the benchmark with the refined prompt increases our Correctness score to **84%** (14 passed, 1 failed).

To verify determinism, we execute a second run on unchanged code:

- **Run 1 Score:** 84% (14 passed, 1 failed on Test Case 7).
- **Run 2 Score:** 83% (14 passed, 1 failed on Test Case 7).

The score remains stable across runs, failing on the exact same test case, confirming evaluation determinism.

Metric 2: Completeness Evaluation

Completeness evaluates whether the chatbot addresses every sub question in a multi part query.

```
# Completeness Metric Definition
completeness_metric = GEval(
    name="Completeness",
    evaluation_steps=[
        "Identify all distinct parts and sub-questions within input query.",
        "Determine whether actual output provides answers for every identified
part.",
        "Penalize partial answers that address initial sub-questions but omit
trailing parts."
    ],
    rubric=[
        Rubric(score_range=(0, 4), explanation="Misses primary components of multi-
part question."),
        Rubric(score_range=(5, 8), explanation="Covers main question but omits
minor sub-parts."),
        Rubric(score_range=(9, 10), explanation="Completely answers all sub-
questions thoroughly.")
    ],
    evaluation_params=[
        LLMTestCaseParams.INPUT,
        LLMTestCaseParams.ACTUAL_OUTPUT,
        LLMTestCaseParams.EXPECTED_OUTPUT
    ],
    model="gpt-4o-mini",
    threshold=0.70,
    strict_mode=False
)
```

Completeness Benchmark Result & System Fix

- **Iteration 1 Result: 68% Score** (5 passed, 10 failed).
- **Root Cause:** Inspection reveals that our generator system prompt instructed the model to "answer concisely". This over constrained the generator, causing it to drop secondary sub questions.

Generator System Prompt Refinement

We update the RAG chatbot generator system prompt to encourage complete responses:

```
Refined Generator Prompt:
"Answer thoroughly. Identify every distinct part of the question and cover each
one.
Include all relevant points the context provides for answering it.
If the question has multiple parts or components, address all of them rather than
stopping at the first."
```

Re-testing the benchmark increases the Completeness score from **68% to 75%**, with passed cases rising from **5 to 14** (1 failed).

Metric 3: Style Evaluation

Style evaluates whether the chatbot's output matches the **CampusX teaching style**. Style evaluation does not require a golden answer; it compares the actual output directly against our brand rubric.

```
# Brand Style Metric Definition
style_metric = GEval(
    name="Style",
    evaluation_steps=[
        "Reward intuitive explanatory tone in plain language.",
        "Verify intuition is explained BEFORE introducing formulas or jargon.",
        "Reward direct conversational register addressing student directly.",
        "An analogy or concrete example is a BONUS when concept is abstract, but a
clear, direct explanation is fully acceptable."
    ],
    rubric=[
        Rubric(score_range=(0, 4), explanation="Dry, stiff, jargon-heavy, or
robotic textbook tone."),
        Rubric(score_range=(5, 8), explanation="Reasonably clear but somewhat
formal or flat."),
        Rubric(score_range=(9, 10), explanation="Authentic CampusX voice:
intuitive, conversational, explaining intuition before formalization.")
    ],
    evaluation_params=[
        LLMTestCaseParams.INPUT,
        LLMTestCaseParams.ACTUAL_OUTPUT
    ],
    model="gpt-4o-mini",
    threshold=0.70,
    strict_mode=False
)
```

Style Iteration Results & Over-Correction Fix

- **Iteration 1 Score: 54%** (9 passed, 6 failed).
- **Root Cause & Fix:** The initial generator prompt contained no tone instructions, yielding textbook-like responses. Furthermore, our initial evaluation rubric strictly penalized answers that lacked analogies. The judge over-corrected and failed valid answers where analogies were unnecessary.

We updated the generator system prompt with explicit style instructions ("explain intuition first in plain conversational language") and adjusted the rubric to treat analogies as a bonus rather than a requirement.

This refinement raised the Style evaluation score from **54% to 74%**.

Summary of Quality Evaluation Results

APPLICATION QUALITY EVALUATION SUMMARY			
Metric Applied	Initial Score	Refined Score	Primary Engineering Lever
Correctness	66% (8 Pass)	84% (14 Pass)	G-Eval custom steps & rubric tuning
Completeness	68% (5 Pass)	75% (14 Pass)	Generator system prompt engineering
Style over-correction fix	54% (0 Pass)	74% (9 Pass)	Generator prompt & rubric

System prompt engineering and evaluation rubric design are primary levers for optimizing RAG performance. By combining G-Eval with explicit evaluation steps, scoring rubrics, and logprob score calculations, we establish a deterministic Application Quality Evaluation Suite.